

```
In [1]: import sys
        !"{sys.executable}" -m pip install numpy pandas matplotlib seaborn
```

Requirement already satisfied: numpy in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (2.5.3)
 Requirement already satisfied: pandas in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.0.5)
 Requirement already satisfied: matplotlib in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.11.1)
 Requirement already satisfied: seaborn in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (0.13.2)
 Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.9.0.post0)
 Requirement already satisfied: tzdata in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2026.3)
 Requirement already satisfied: contourpy>=1.0.1 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.3.3)
 Requirement already satisfied: cycler>=0.10 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.28.2 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (4.64.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.5.1)
 Requirement already satisfied: packaging>=20.0 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (26.3)
 Requirement already satisfied: pillow>=9 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (12.3.0)
 Requirement already satisfied: pyparsing>=3 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (3.3.2)
 Requirement already satisfied: six>=1.5 in C:\Users\Anant Mishra\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

```
In [2]: import warnings
        warnings.filterwarnings('ignore')
```

```
In [3]: #import libraries
        import numpy as np
        import pandas as pd
        import matplotlib.pyplot as plt
        import seaborn as sns
        import sqlite3
```

```
In [4]: #1. import database/data
```

```
In [5]: conn = sqlite3.connect('customer_churn.db')

        sql_query = """
                SELECT name
                From  sqlite_master
                WHERE type='table';

        """
```

```

tables = pd.read_sql(sql_query,conn)

#create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()

```

Created dataframe: df_db_customer

Created dataframe: df_db_subscription

Created dataframe: df_db_support

```

In [6]: # Print table names and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()

```

Table Name: db_customer

Columns:

['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription

Columns:

['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support

Columns:

['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

```

In [7]: #2 data cleaning

```

```

In [8]: df_db_customer.head()
df_db_customer.tail()

```

```
Out[8]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead>
<tbody> <tr> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody>
<tbody></tbody>
```

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
In [9]: df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

```
In [10]: # a. rename col - name
# b. drop columns - interest and pincode
# c. change data type - dob
# d. data standarization - gender
# e. fix missing values - country
```

```
In [11]: # a. rename col - name

df_db_customer.rename(columns ={'name' : 'customer_name'}, inplace= True)
df_db_customer
```

```
Out[11]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead>
<tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th>
<th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th>
<th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th>
<th>18</th> <th>19</th> <th>20</th> </tbody> <tbody> </tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	None
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	None
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	None
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	None
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	None
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	None
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	None
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	None
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	None
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	None
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	None

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
In [12]: # b. drop columns - interest and pincode
#df_db_customer.drop(df_db_customer.columns[-2:], axis=1)
#df_db_customer.columns[6:], axis=1)

df_db_customer.drop(columns=['interests','pincode'], inplace=True)
```

```
In [13]: df_db_customer.info()

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      21 non-null    str
1   customer_name   21 non-null    str
2   country         18 non-null    str
3   state           21 non-null    str
4   gender          21 non-null    str
5   dob             21 non-null    str
dtypes: str(6)
memory usage: 1.1 KB
```

```
In [14]: # c. change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

```
In [15]: # d. data standarization - gender

#df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Women
```

```
In [16]: df_db_customer['gender'].unique()
```

```
Out[16]: <StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

```
In [17]: # e. fix missing values - country

df_db_customer[df_db_customer['country'].isna()]
```

```
Out[17]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th> <th>8</th>
<th>12</th> </tbody> <tbody></tbody>
```

```
0013-MHZWF  durga  NaN      Delhi  Female  1988-12-10
```

```
0015-UOCOJ  maya  NaN  Kathmandu  Female  1985-07-07
```

```
0018-NYROU  chitra  NaN  Telangana  Female  2004-12-01
```

```
In [18]: #country and state - uniquw value pair
```

```
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state')
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'])
```

```
In [19]: df_db_customer[df_db_customer['country'].isna()]
```

```
Out[19]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> </thead> <tbody> </tbody> <tbody></tbody>
```

```
In [ ]:
```

```
In [20]: df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    str
1   subscription_start_date 21 non-null    str
2   subscription_type       21 non-null    str
3   renewal_date           21 non-null    str
4   plan_type              21 non-null    str
5   contract_type          21 non-null    str
6   cancellation_date       6 non-null     str
7   cancellation_reason     6 non-null     str
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score            21 non-null    int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

```
In [21]: date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']
df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
```

```
In [22]: df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null     str
1   subscription_start_date              21 non-null     datetime64[us]
2   subscription_type                     21 non-null     str
3   renewal_date                         21 non-null     datetime64[us]
4   plan_type                            21 non-null     str
5   contract_type                        21 non-null     str
6   cancellation_date                    6 non-null      datetime64[us]
7   cancellation_reason                  6 non-null      str
8   monthly_charges                      21 non-null     float64
9   cltv                                 21 non-null     int64
10  churn_score                           21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

```
In [ ]:
```

```
In [23]: df_db_support.head()
```

```
Out[23]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0003-MKNFE  2024-08-28 00:00:00  N  60  None      service issue
0003-MKNFE  2024-08-28 00:00:00  Y  10  None      demanded refund
0013-EXCHZ  2024-01-20 00:00:00  Y  20  None      NaN
0013-MHZWF  2025-03-18 00:00:00  N  90  None      guidance to renew
0013-SMEOE  2024-11-01 00:00:00  N  30  None      NaN
```

```
In [24]: df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            9 non-null     str
1   complaint_date        9 non-null     str
2   escalations           9 non-null     str
3   csat_score            9 non-null     int64
4   col_1                 0 non-null     object
5   comment               4 non-null     str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

```
In [25]: df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

```
In [26]: df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])
df_db_support.info()

<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customerid      9 non-null     str
1   complaint_date  9 non-null     datetime64[us]
2   escalations     9 non-null     str
3   csat_score      9 non-null     int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

```
In [ ]:
```

```
In [27]: #3. Feature Engineering and Data Analytics
```

```
In [28]: #create a new columns using existing col - churn flag
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date']
```

```
In [29]: df_db_subscription.head()
```

Out[29]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>

<thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	0
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	0
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	0
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

```
In [30]: #first fix support table duplicates then merge
df = (df_db_subscription
```

```
.merge(df_db_customer, on = 'customerid' , how= 'left')
.merge(df_db_support, on = 'customerid', how= 'left'))
```

In [31]: df_db_subscription.shape

Out[31]: (21, 12)

In [32]: df.shape

Out[32]: (23, 20)

In [33]: df_db_subscription['customerid'].nunique()

Out[33]: 21

In [34]: df_db_customer['customerid'].nunique()

Out[34]: 21

In [35]: df_db_support['customerid'].nunique()

Out[35]: 7

In [36]: df_db_support['customerid'].size

Out[36]: 9

In [37]: df_db_support

Out[37]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody>
</tbody>

0003-MKNFE 2024-08-28 N 60

0003-MKNFE 2024-08-28 Y 10

0013-EXCHZ 2024-01-20 Y 20

0013-MHZWF 2025-03-18 N 90

0013-SMEOE 2024-11-01 N 30

0017-IUDMW 2024-04-10 Y 25

0019-EFAEP 2024-09-27 Y 30

0022-TCJCI 2024-09-13 Y 10

0022-TCJCI 2024-09-14 N 90

```
In [38]: df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'
```

```
In [39]: df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('custom
```

```
In [40]: #merge df
df = (df_db_subscription
      .merge(df_db_customer, on = 'customerid' , how= 'left')
      .merge(df_db_support, on = 'customerid', how= 'left'))
```

```
In [41]: df.shape
```

```
Out[41]: (21, 21)
```

```
In [42]: df.to_csv('exported_churn_data.csv', index = False)
```

```
In [43]: # Data Analysis
```

```
In [44]: #1. Churn Rate
```

```
In [45]: df.columns
```

```
Out[45]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
               'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
               'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
              dtype='str')
```

```
In [46]: churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

Churn Rate = 28.57 %

```
In [47]: #2 Retention Rate
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

Retention Rate = 71.43 %

```
In [48]: #3 Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean()).mul(100).round(2).res
print(churn_by_plan)
```

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

```
In [49]: #5. ARPU - Avg Revenue per User
arpu = df['monthly_charges'].mean()
print('ARPU = ' , round(arpu,2))
```

ARPU = 18.85

```
In [50]: #6 Avg Customer Tenure
#count of days users has used our service : cancellation date else current date
today= pd.Timestamp.today()

df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) =", round(avg_tenure,0))
```

Avg Tenure (Days) = 1536 0

```
In [51]: #7 Revenue at risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs Lakhs) =", revenue_at_risk)
```

Revenue at Risk (Rs Lakhs) = 73.94

```
In [52]: #8 Escalation Rate
escalation_rate=(df['escalations']=='Y').mean()*100
print("Escalation rate =", round(escalation_rate,2),"%")
```

Escalation rate = 19.05 %

```
In [53]: #9 Average Complaint Rate per user
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg complaints Per user = ", round(avg_complaints, 2))
```

Avg complaints Per user = 0.43

```
In [54]: #10 Coorelation Escalation vs churn
df['escalations'] = np.where(df['escalations'] == 'Y' , 1, 0) # encoding string to

corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(corr_df['churn_flag'])
print("Correlation between escalation vs churn is = ", round(correlation, 2))
```

Correlation between escalation vs churn is = 0.77

```
In [55]: #11 churn Risk - create a column using existing tool

conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']

df['churn_risk'] =np.select(conditions,choices,default='unknown')
```

In [56]: `df[['churn_risk', 'churn_score']].head()`

Out[56]: `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>churn_risk</th> <th>churn_score</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>`

low 12

high 91

low 34

low 8

high 88

In [57]: *#4. Visualization using Matplotlib*

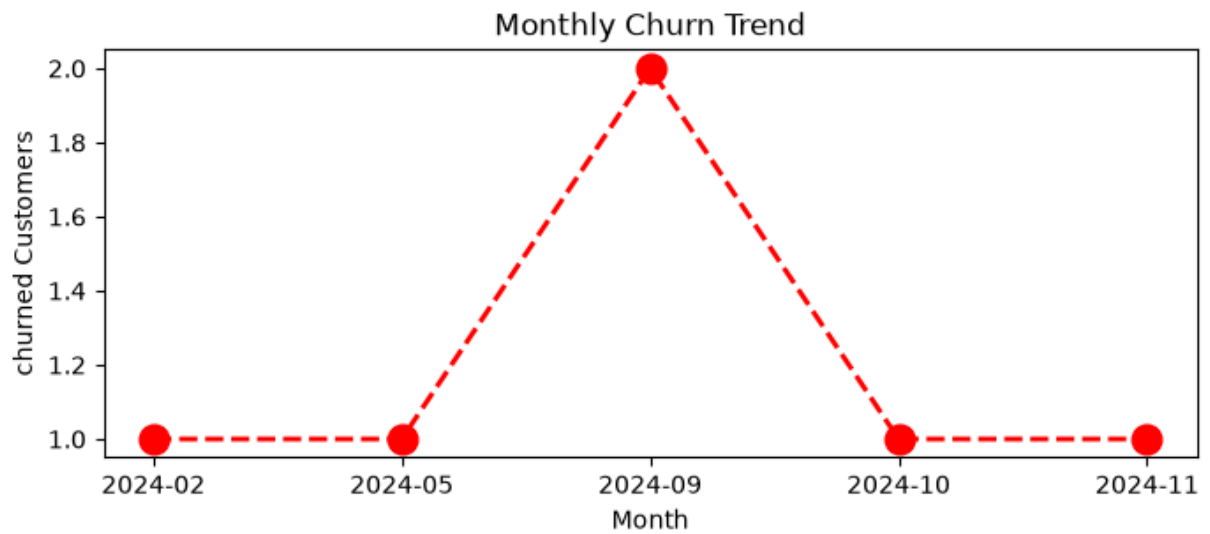
In [58]: `df_visual = df.copy()`

In [59]: `df_visual.columns`

Out[59]: `Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'complaint_date', 'escalations', 'csat_score', 'complaint_count',
'tenure_days', 'churn_risk'],
dtype='str')`

In [60]: *#4.1 Monthly Churn Trend (Time Series KPI)*

```
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')
churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month')
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='red', marker='o')
plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('churned Customers')
plt.show()
```



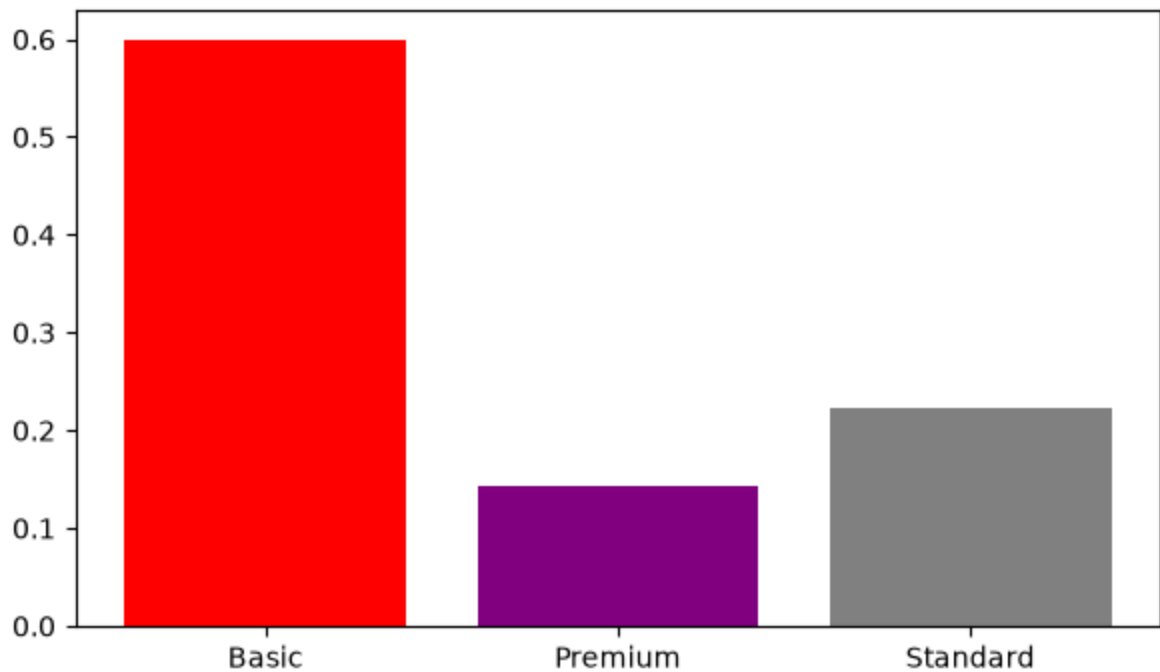
```
In [61]: #4.2 Churn by Plan Type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

colors = ['Red' , 'purple' , 'grey']    #Palette

plt.figure(figsize=(7,4))

plt.bar(churn_plan.index,churn_plan.values, color = colors)

plt.show()
```



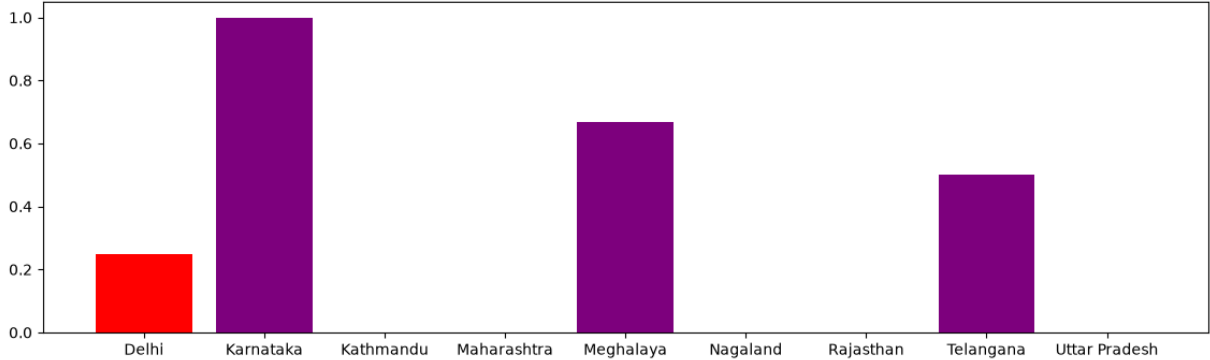
```
In [62]: #4.2 Churn by State
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

colors = ['Red' , 'purple' , 'grey']    #Palette

plt.figure(figsize=(14,4))
```

```
plt.bar(churn_plan.index,churn_plan.values, color = colors)

plt.show()
```



In [63]: *#Visualisation using Seaborn*

In [64]: *# encoding - convert string to numeric so that we can find corr between features*
df_visual.columns

Out[64]: Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'tenure_days', 'churn_risk', 'cancellation_month'], dtype='str')

In [65]: df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'escalation

Out[65]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }.dataframe tbody tr th { vertical-align: top; }.dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>escalations</th> <th>churn_risk</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>

Standard	Annual	12	0	0	low
Premium	Annual	91	1	1	high
Basic	Monthly	34	0	0	low
Premium	Annual	8	0	0	low
Standard	Monthly	88	1	1	high

In [66]: *#incorrect method of encoding - as numbers are not assigned based on priority*

```
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag',
categorical_cols = ['plan_type', 'contract_type', 'churn_risk']
```

```
for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

In [67]: `df_encoded.head()`

Out[67]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>escalations</th> <th>churn_risk</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>

```
2  0  12  0  0  1
```

```
1  0  91  1  1  0
```

```
0  1  34  0  0  1
```

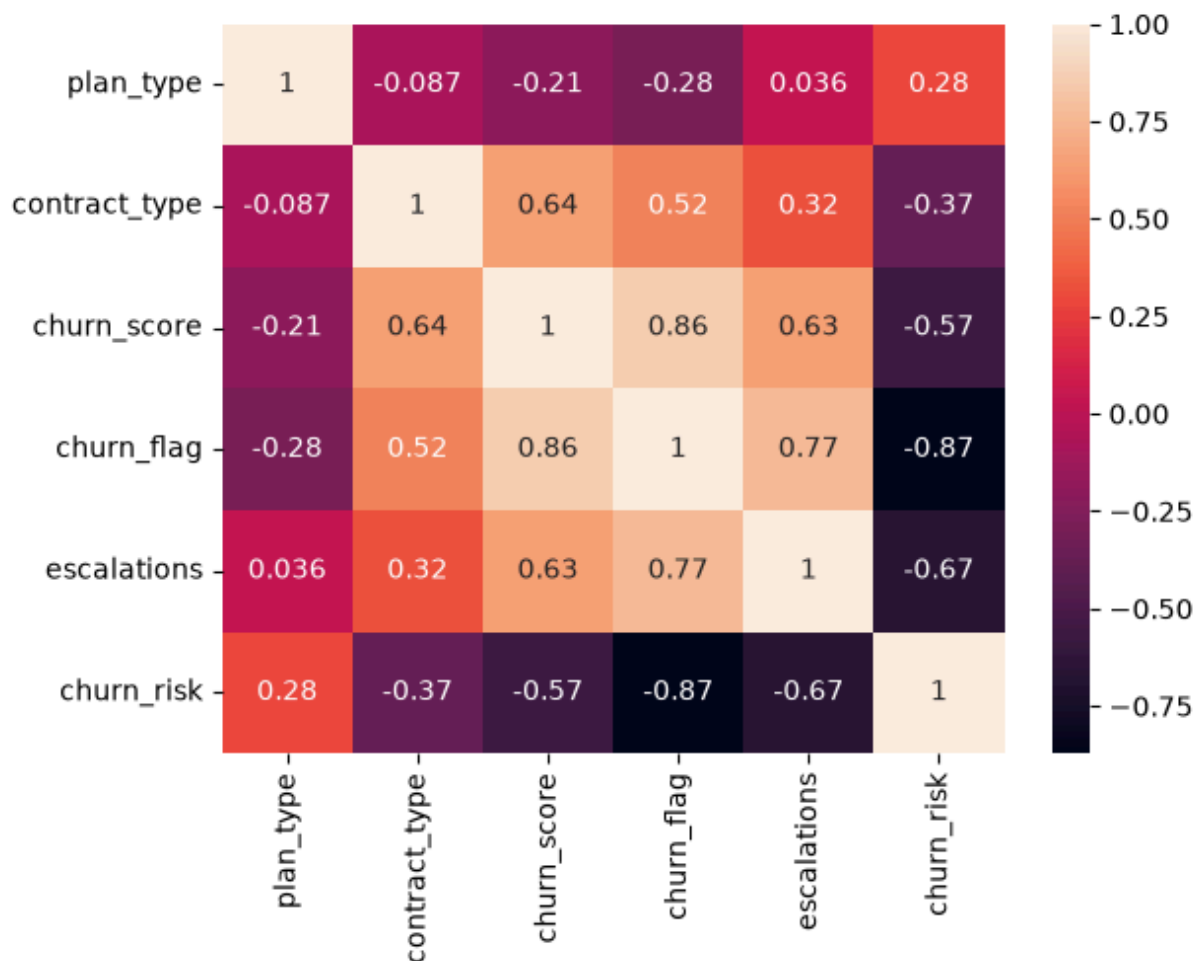
```
1  0   8  0  0  1
```

```
2  1  88  1  1  0
```

In [68]: *#Heatmap (correlation Matrix)*

```
sns.heatmap(df_encoded.corr(), annot=True)
```

Out[68]: <Axes: >



```
In [69]: df_visual['plan_type'].unique()
```

```
Out[69]: <StringArray>
['Standard', 'Premium', 'Basic']
Length: 3, dtype: str
```

```
In [70]: #correct method of encoding - as numbers are not assigned based on priority
```

```
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag' ,
order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annul'],
    'churn_risk' : ['low' , 'med' , 'high']
}

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories
```

```
In [71]: df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag' , 'escalation
```

Out[71]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>escalations</th> <th>churn_risk</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>

Standard	Annual	12	0	0	low
----------	--------	----	---	---	-----

Premium	Annual	91	1	1	high
---------	--------	----	---	---	------

Basic	Monthly	34	0	0	low
-------	---------	----	---	---	-----

Premium	Annual	8	0	0	low
---------	--------	---	---	---	-----

Standard	Monthly	88	1	1	high
----------	---------	----	---	---	------

In [72]: df_encoded.head()

Out[72]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>escalations</th> <th>churn_risk</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>

1	-1	12	0	0	0
---	----	----	---	---	---

2	-1	91	1	1	2
---	----	----	---	---	---

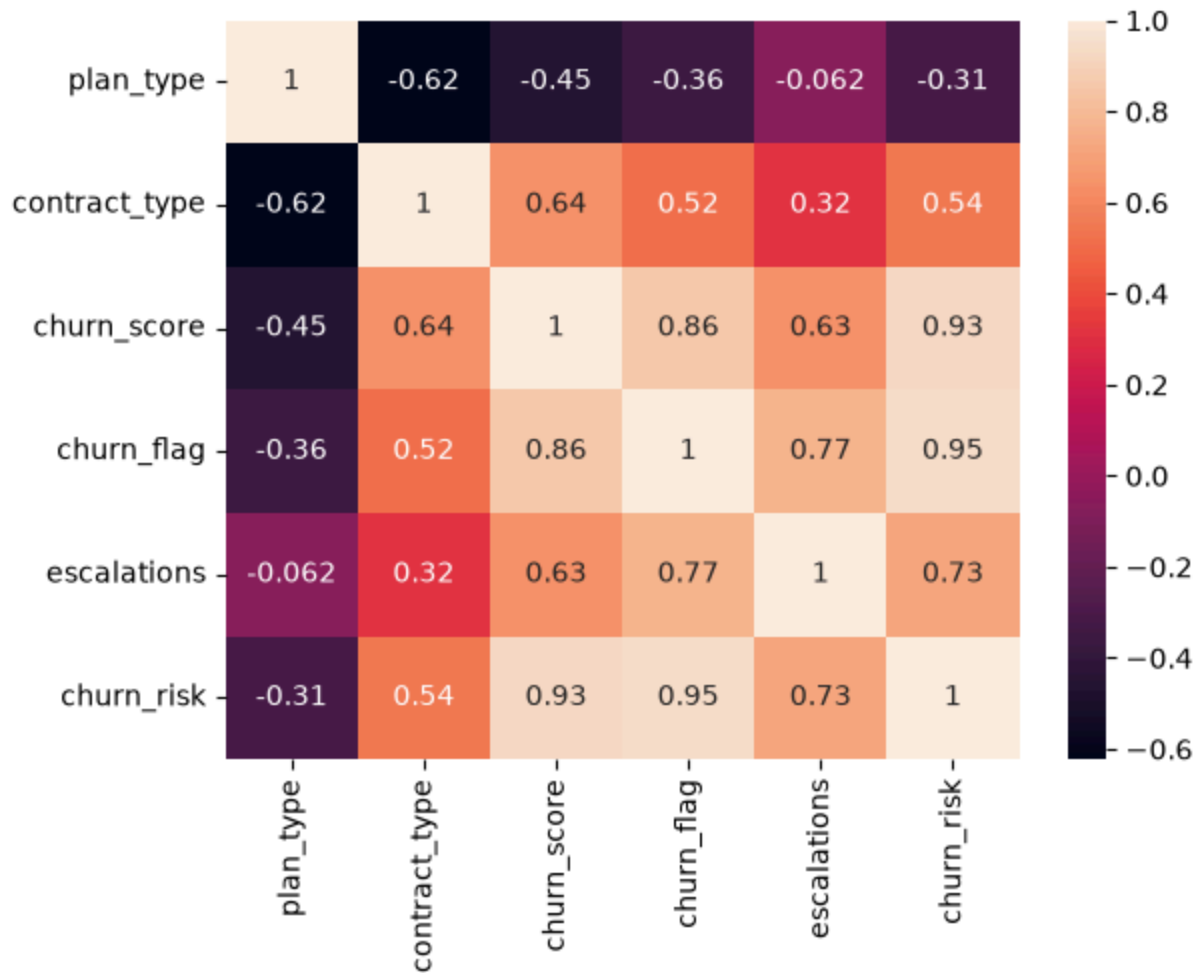
0	0	34	0	0	0
---	---	----	---	---	---

2	-1	8	0	0	0
---	----	---	---	---	---

1	0	88	1	1	2
---	---	----	---	---	---

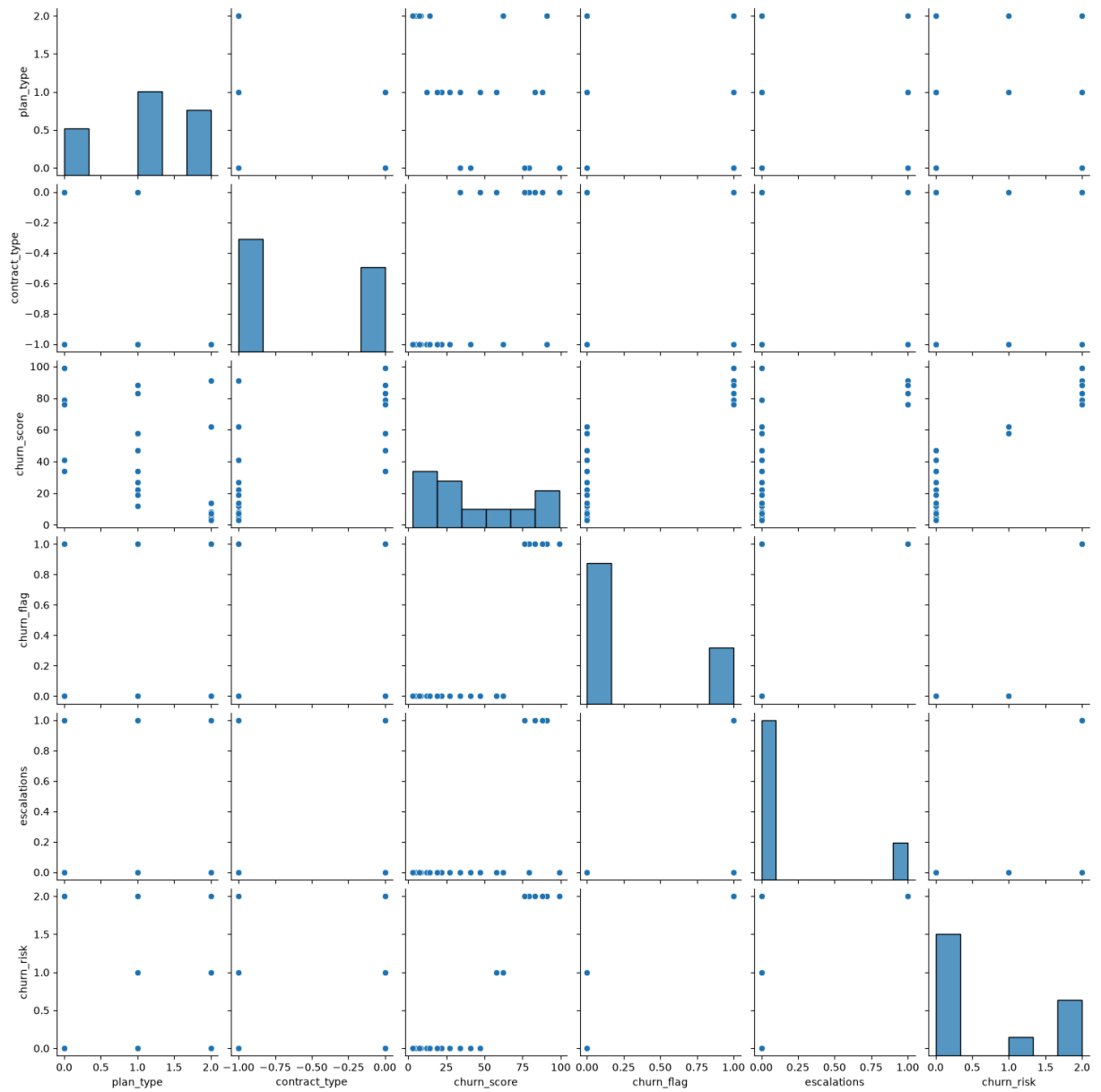
In [73]: *#Heatmap (correlation Matrix)*
sns.heatmap(df_encoded.corr(), annot=True)

Out[73]: <Axes: >



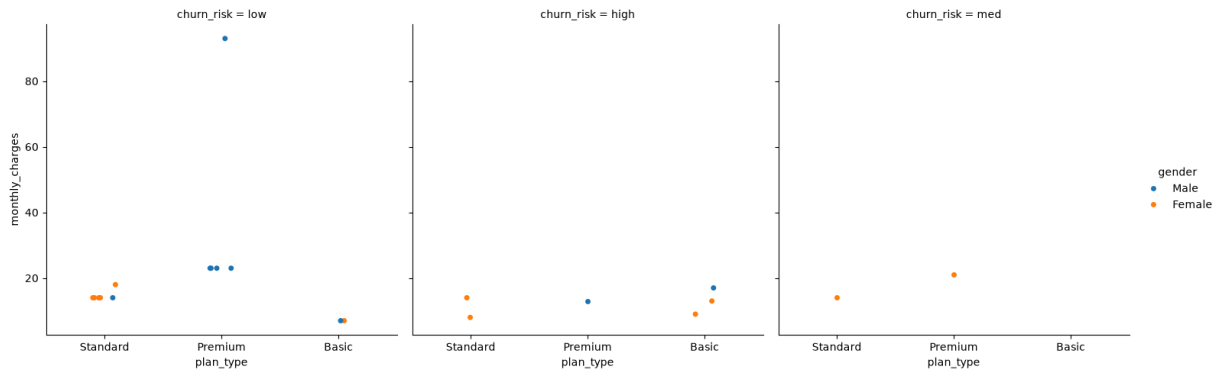
```
In [74]: #pairplot using seaborn - to determine relationship between databses
sns.pairplot(df_encoded)
```

```
Out[74]: <seaborn.axisgrid.PairGrid at 0x29bc346c440>
```



```
In [75]: #catplt/facefrid plot - multi-dim comparision
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk')
```

```
Out[75]: <seaborn.axisgrid.FacetGrid at 0x29bc5f96e40>
```



In [76]: `#pivot table`

```
In [77]: pd.pivot_table(
df_visual,
values='churn_flag',
index='plan_type',
aggfunc='mean'
).reset_index()
```

Out[77]: `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>churn_flag</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> </tbody> <tbody></tbody>`

Basic 0.600000

Premium 0.142857

Standard 0.222222

In [78]: `df_visual.columns`

Out[78]: `Index(['customerid', 'subscription_start_date', 'subscription_type',
'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
'complaint_date', 'escalations', 'csat_score', 'complaint_count',
'tenure_days', 'churn_risk', 'cancellation_month'],
dtype='str')`

```
In [79]: pd.pivot_table(
df_visual,
values=['monthly_charges', 'customerid', 'churn_flag'],
index='plan_type',
aggfunc = {
'monthly_charges' : 'sum',
'customerid' : 'nunique',
'churn_flag' : 'mean',
}
)
```

```
Out[79]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th>
<th>plan_type</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Basic</th>
<th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000  5   52.95
0.142857  7  218.93
0.222222  9  123.91
```

```
In [80]: #working with sql in Python (Pandas)
```

```
In [81]: import sqlite3

conn = sqlite3.connect("test_database.sqlite")
#create table
conn.execute("""
    CREATE TABLE IF NOT EXISTS users (
        first_name TEXT,
        country TEXT,
        budget INTEGER
    )
""")
#save and commit
conn.commit()
```

```
In [82]: #insert data

cursor = conn.cursor()
#write sql queries to insert records in sql table
cursor.execute("""
    INSERT INTO users VALUES
        ('ANANT', 'India', 5000),
        ('RISHABH', 'Germany', 2500),
        ('ADITYA', 'India', 3500)
""")
#commit and save
conn.commit()

print("Data Inserted Successfully")
```

Data Inserted Successfully

```
In [83]: #check inserted data in table

conn = sqlite3.connect("test_database.sqlite")
query= """SELECT * FROM users"""

df_results = pd.read_sql(query,conn)

df_results.head()
```

Out[83]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>first_name</th> <th>country</th> <th>budget</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody>
</tbody>

ANANT	India	5000
RISHABH	Germany	2500
ADITYA	India	3500
ANANT	India	5000
RISHABH	Germany	2500

In [84]: *#aggregation*
 query = """
 SELECT country , sum(budget) as total_budget
 From users
 GROUP BY country
 """"
 df_agg = pd.read_sql(query,conn)
 df_agg

Out[84]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>country</th> <th>total_budget</th> </thead> <tbody> <th>0</th>
<th>1</th> </tbody> <tbody></tbody>

Germany	25000
India	85000

In [85]: *#always close the conn with db once the task is over*
 conn.close()

In []: